

UNITÀ
01

LEZIONE
01

Introduzione a Python

Oggetti, istruzioni e condizioni

Prof. Francesco Gobbi

Anno scolastico 2026/2027

I.I.S.S. Galileo Galilei – Ostiglia (MN)

Che cos'è Python

Python è un **linguaggio di programmazione ad alto livello**: consente di esprimere algoritmi senza gestire direttamente i dettagli dell'hardware.

Esecuzione L'interprete esegue le istruzioni del programma.

Ambiti d'uso Automazione, analisi dei dati, applicazioni web e intelligenza artificiale.

Modello dei dati Ogni dato è rappresentato da un oggetto, che ha un'identità, un tipo e un valore.

Tipizzazione dinamica: il tipo appartiene all'oggetto. Un nome può essere associato prima a un numero e poi a una stringa, senza dichiararne il tipo.

Notebook Jupyter: codice e risultati

Un notebook è un documento `.ipynb` composto da **celle**.

- ▶ Le celle di **testo** contengono spiegazioni e appunti.
- ▶ Le celle di **codice** contengono istruzioni Python.
- ▶ Il **kernel** è il processo che esegue il codice e conserva gli oggetti della sessione.
- ▶ Il risultato compare sotto la cella. **Shift + Invio** esegue la cella e passa alla successiva.

Conta l'**ordine di esecuzione**, non solo la posizione delle celle. Dopo un riavvio del kernel, rieseguiamo le celle dall'alto.

Visual Studio Code o Google Colab

Visual Studio Code

1. Installiamo Python e VS Code sul computer.
2. Aggiungiamo le estensioni Microsoft **Python** e **Jupyter**.
3. Creiamo un file `lezione.ipynb`.
4. Selezioniamo un kernel Python e installiamo `ipykernel` se richiesto.

Google Colab

1. Apriamo **`colab.research.google.com`** e accediamo con un account Google.
2. Creiamo un nuovo notebook.
3. Colleghiamo la sessione ed eseguiamo una cella.

Python viene eseguito su una macchina remota, senza installazione locale.

Documentazione: Notebook in VS Code Google Colab

Classe, oggetto, attributi e metodi

Concetto	Significato	Esempio: una bicicletta
Classe	Definisce un tipo di oggetti e i comportamenti disponibili.	Bicicletta è il modello comune.
Oggetto	È una specifica istanza della classe, con una propria identità.	La bicicletta di Anna.
Attributo	È un dato associato all'oggetto o alla classe.	colore = "rosso", velocità attuale.
Metodo	È un'operazione che possiamo chiamare su un oggetto.	frena() riduce la velocità.

Due biciclette possono appartenere alla stessa classe e avere uno **stato diverso**, per esempio colori o velocità differenti.

Le prime righe in Python

```
# Un commento spiega il codice  
nome = "Anna"  
eta = 16  
print("Ciao", nome)  
print(eta + 1)
```

OUTPUT Ciao Anna
 17

- ▶ # introduce un commento, che Python ignora.
- ▶ = assegna a un nome il riferimento a un oggetto.
- ▶ print() mostra un risultato. Le stringhe richiedono le virgolette.
- ▶ Maiuscole e minuscole contano: nome e Nome sono diversi.

Mostrare un risultato con print()

```
nome = "Anna"  
punti = 12  
print("Ciao", nome)  
print("Punti:", punti)  
print(2 + 3)  
print("2 + 3")
```

```
OUTPUT  Ciao Anna  
        Punti: 12  
        5  
        2 + 3
```

print() mostra i valori passati tra parentesi.

- ▶ Le virgole separano gli argomenti. Di norma l'output li separa con uno spazio.
- ▶ Ogni chiamata termina andando a capo.
- ▶ $2 + 3$ viene calcolato.
- ▶ "2 + 3" è testo e viene mostrato così com'è.

Leggere un valore con input()

```
nome = input("Come ti chiami? ")  
print("Ciao", nome)
```

```
SESSIONE Come ti chiami? Anna  
Ciao Anna
```

1. `input()` mostra la domanda e aspetta la risposta.
2. Scriviamo Anna e premiamo Invio: la stringa restituita viene associata a `nome`.
3. `print()` usa quel valore per costruire il saluto.

`input()` restituisce sempre una **stringa**, anche se digitiamo delle cifre. Nel notebook la cella resta in attesa della risposta.

Gli oggetti di base

Classe	Esempi	Significato
<code>int</code>	12, -3, 0	Numeri interi.
<code>float</code>	3.5, -0.2	Numeri in virgola mobile: possono approssimare valori decimali.
<code>bool</code>	True, False	Valori di verità. L'iniziale è maiuscola.
<code>str</code>	"ciao", "12"	Testo racchiuso tra virgolette.

12 è un numero, "12" è una stringa. Il tipo determina le operazioni possibili.
`type(oggetto)` permette di verificarlo.

Convertire il testo in un numero

```
testo = input("Eta: ")
eta = int(testo)
print(eta + 1)
print(testo + "1")
```

```
SESSIONE Eta: 16
          17
          161
```

`input()` restituisce "16".

`int(testo)` restituisce l'intero 16. La conversione lascia invariata la stringa di partenza.

`eta + 1` somma due numeri.

`testo + "1"` unisce due stringhe.

`float("2.5")` restituisce 2.5. Per i decimali usiamo il punto. Un testo non valido, come `int("ciao")`, provoca `ValueError`.

Chiamare un metodo di una stringa

```
nome = "Anna"  
maiuscolo = nome.upper()  
print(maiuscolo)  
print(nome)
```

OUTPUT ANNA
 Anna

Oggetto: la stringa "Anna", di tipo str, a cui si riferisce nome.

Metodo: upper() restituisce una stringa con le lettere maiuscole.

Risultato: maiuscolo si riferisce a "ANNA". La stringa iniziale resta "Anna".

Le stringhe sono **immutabili**. La forma oggetto.metodo() chiama un metodo. La forma print(...) chiama invece una funzione.

Operazioni con i numeri

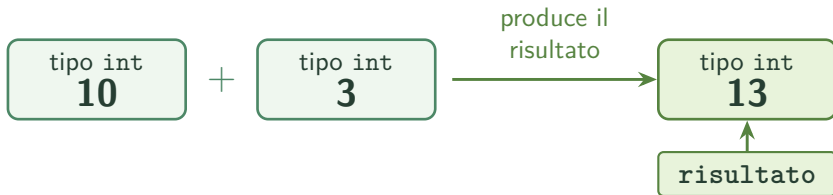
Operatore	Operazione	Esempio e risultato
+, -	Somma e sottrazione	7 + 2 dà 9
*	Moltiplicazione	7 * 2 dà 14
/	Divisione	7 / 2 dà 3.5
//	Divisione arrotondata per difetto	7 // 2 dà 3
%	Resto	7 % 2 dà 1
**	Potenza	2 ** 3 dà 8

Precedenza: 2 + 3 * 4 dà 14, mentre (2 + 3) * 4 dà 20.

6 / 2 restituisce il float 3.0. Con i negativi, -7 // 2 dà -4. Alcuni decimali hanno una rappresentazione approssimata come float.

Il risultato di un'operazione è un oggetto

```
a = 10  
b = 3  
risultato = a + b
```



I numeri sono immutabili: la somma non modifica 10 e 3. Il nome `risultato` viene associato all'oggetto di valore 13.

Python può riutilizzare un oggetto immutabile già esistente con lo stesso tipo e valore del risultato.

Più nomi, uno stesso oggetto

```
a = 10  
b = a  
c = a  
print(a, b, c)
```

OUTPUT 10 10 10



`b = a` associa a `b` il riferimento già associato ad `a`. Lo stesso avviene per `c`.

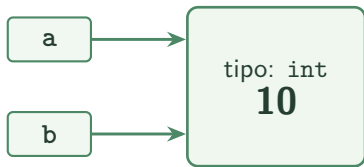
Ogni freccia è un **riferimento**. L'assegnazione aggiunge un'associazione tra un nome e un oggetto: **non copia automaticamente l'oggetto**.

Il nome è un'etichetta, non un puntatore da manipolare come un indirizzo di memoria.

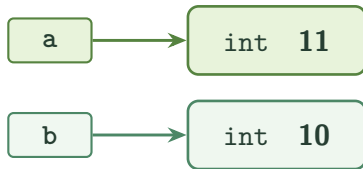
Riassegnazione: il nome cambia riferimento

```
a = 10  
b = a  
a = a + 1  
print(a, b)  # 11 10
```

PRIMA: a e b condividono l'oggetto



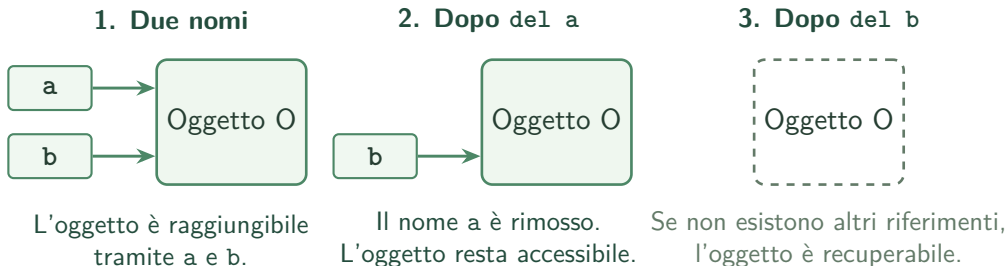
DOPO: solo a cambia riferimento



Prima si valuta $a + 1$ usando il valore 10. **Poi** si associa a al risultato 11. L'oggetto 10 resta invariato e b continua a riferirsi a esso.

Garbage collection: recuperare memoria

Quando un oggetto diventa **irraggiungibile**, il sistema di gestione della memoria può recuperare lo spazio che occupa.



del elimina l'associazione del nome. **Non ordina di distruggere l'oggetto.** Il recupero della memoria non è necessariamente immediato.

Confronti e condizioni

Un confronto tra i valori presentati restituisce un booleano: True oppure False.

<code>==</code>	uguale a
<code>!=</code>	diverso da
<code><, ></code>	minore, maggiore
<code><=, >=</code>	minore o uguale, maggiore o uguale

```
eta = 16
print(eta >= 18)
print(eta == 16)
```

```
OUTPUT  False
         True
```

`=` serve per assegnare. `==` serve per confrontare i valori.

Solo if: eseguire un blocco se serve

```
temperatura = 32
if temperatura > 30:
    print("Fa caldo")
print("Fine controllo")
```

Con temperatura = 32

OUTPUT Fa caldo
Fine controllo

Se la condizione è vera, Python esegue il blocco indentato. Se è falsa, lo salta.

`print("Fine controllo")` è fuori dal blocco: viene eseguito in entrambi i casi.

Con temperatura = 20

OUTPUT Fine controllo

I due punti introducono il blocco. L'**indentazione** ne delimita le istruzioni: usiamo quattro spazi per ogni livello.

if ed else: due alternative

```
eta = 16
if eta >= 18:
    print("Maggiorenne")
else:
    print("Minorenne")
```

if gestisce il caso vero.

else gestisce il caso falso e non richiede una condizione.

A seconda del valore della condizione, viene eseguito **uno solo** dei due blocchi.

Età	Valore di eta >= 18	Output
16	False	Minorenne
18	True	Maggiorenne
21	True	Maggiorenne

if, elif ed else: più alternative

```
voto = 7
if voto >= 8:
    print("Ottimo")
elif voto >= 6:
    print("Sufficiente")
else:
    print("Da migliorare")
```

Le condizioni vengono controllate **dall'alto verso il basso**.

Al primo risultato vero, Python esegue quel blocco e salta il resto della catena.

else interviene quando tutte le condizioni precedenti sono false.

Esempi: con 9 otteniamo Ottimo, con 7 Sufficiente, con 4 Da migliorare.

L'ordine conta: se controllassimo prima `voto >= 6`, anche 9 entrerebbe in quel ramo. Due if separati possono invece eseguire entrambi i blocchi.

Operatori logici: and, or, not

Combinano o negano condizioni. Qui A e B sono booleani.

A	B	A and B	A or B	not A
False	False	False	False	True
False	True	False	True	True
True	False	False	True	False
True	True	True	True	False

and: entrambe vere. or: almeno una vera. not: nega.

Tra operatori logici, la precedenza è not, poi and, poi or. Le parentesi rendono esplicito il raggruppamento.

Operatori logici: un esempio spiegato

```
eta = 16
ha_permesso = True
sospeso = False
accesso = (eta >= 18 or ha_permesso) and not sospeso
print(accesso)
```

OUTPUT True

1. `eta >= 18` è falso, ma `ha_permesso` è vero: il risultato di `or` è `True`.
2. `not sospeso` vale `True`, perché `sospeso` è falso.
3. `True and True` vale `True`: l'accesso è consentito.

Con `sospeso = True`, il risultato finale sarebbe `False`.

Esercizio 1: il prezzo scontato

Testo

Leggi il prezzo di un prodotto. Se il prezzo è almeno 50 euro, applica uno sconto del 10%. Mostra sempre il prezzo finale.

Dati ammessi

Un numero non negativo. Se contiene decimali, separali con il punto.

Esempi attesi

- ▶ Prezzo 80: il risultato è 72.0 euro.
- ▶ Prezzo 30: il risultato è 30.0 euro.
- ▶ Prezzo 50: il risultato è 45.0 euro.

Quale conversione serve dopo `input()`? È necessario un ramo `else`?

Esercizio 1: soluzione commentata

```
# Leggiamo un numero anche decimale
prezzo = float(input("Prezzo: "))
if prezzo >= 50:
    # Paghiamo il 90% del prezzo iniziale
    prezzo = prezzo * 0.9
# Questa istruzione vale per entrambi i casi
print("Prezzo finale:", prezzo)
```

```
SESSIONE Prezzo: 80
          Prezzo finale: 72.0
```

else non serve: sotto 50 euro il riferimento di prezzo resta invariato. Con 80 euro, il calcolo produce il risultato 72.0 e il nome viene riassegnato.

Esercizio 2: numero pari o dispari

Testo

Leggi un numero intero e comunica se è pari oppure dispari.

Criterio

Un intero è pari quando il resto della divisione per 2 è zero.

Esempi attesi

- ▶ 8: Pari.
- ▶ 7: Dispari.
- ▶ 0: Pari.

Usa `int()`, l'operatore `%` e una struttura `if-else`.

Esercizio 2: soluzione commentata

```
numero = int(input("Numero: "))  
# Confrontiamo il resto con zero  
if numero % 2 == 0:  
    print("Pari")  
else:  
    print("Dispari")
```

```
SESSIONE Numero: 7  
Dispari
```

Con 7, il resto è 1. Il confronto `1 == 0` è falso, quindi viene eseguito `else`.

Con 8, il resto è 0: viene eseguito il primo ramo.

I due casi sono alternativi e coprono tutti i numeri interi, compreso lo zero.

Esercizio 3: valutare un voto

Testo

Leggi un voto intero e mostra il messaggio corrispondente.

Valore inserito	Messaggio
Minore di 1 oppure maggiore di 10	Voto non valido
Da 8 a 10	Ottimo
Da 6 a 7	Sufficiente
Da 1 a 5	Da migliorare

Esempi: 11 non è valido, 8 è ottimo, 6 è sufficiente.

Controlla prima i valori fuori intervallo usando `or`. Poi ordina le soglie dalla più alta alla più bassa.

Esercizio 3: soluzione commentata

```
voto = int(input("Voto: "))  
# Basta violare uno dei limiti  
if voto < 1 or voto > 10:  
    print("Voto non valido")  
elif voto >= 8:  
    print("Ottimo")  
elif voto >= 6:  
    print("Sufficiente")  
else:  
    print("Da migliorare")
```

Con 7, il voto è valido e non raggiunge 8. Il terzo controllo è vero: otteniamo Sufficiente.

Con 11, il primo controllo è vero e la catena termina subito.

Nell'ultimo ramo restano solo i voti da 1 a 5: gli altri casi sono già stati esclusi.

Qui assumiamo che l'utente inserisca un intero. Un testo come "sette" richiede una gestione degli errori, che vedremo più avanti.